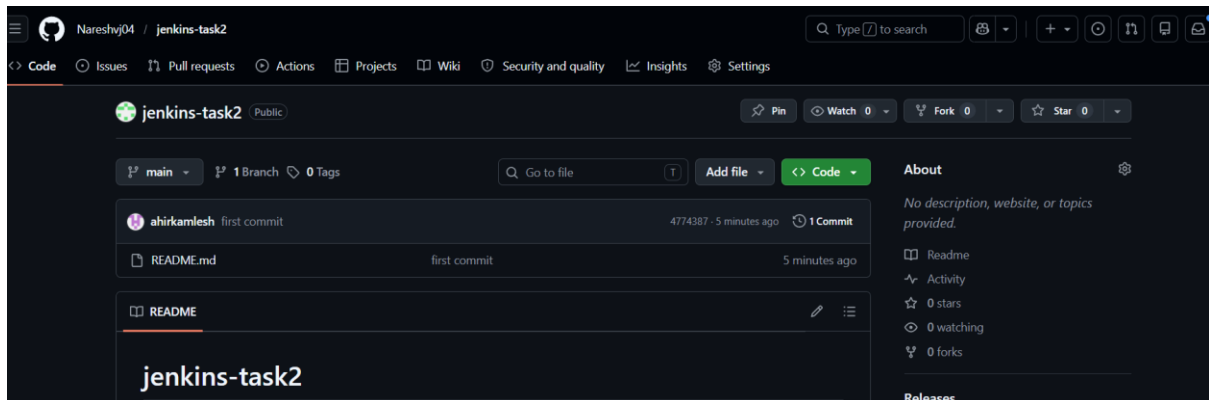


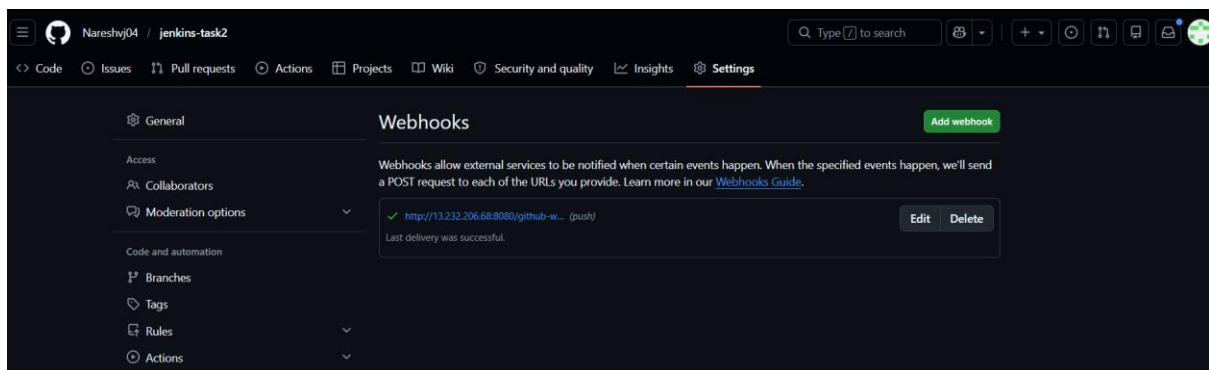
Jenkins Task-2

Create a simple script file and push it to repo. Create a project in Jenkins connected to your GitHub repository. When a commit is made to your repo, automatically build must get triggered from Jenkins and the output must be shared to me via email.

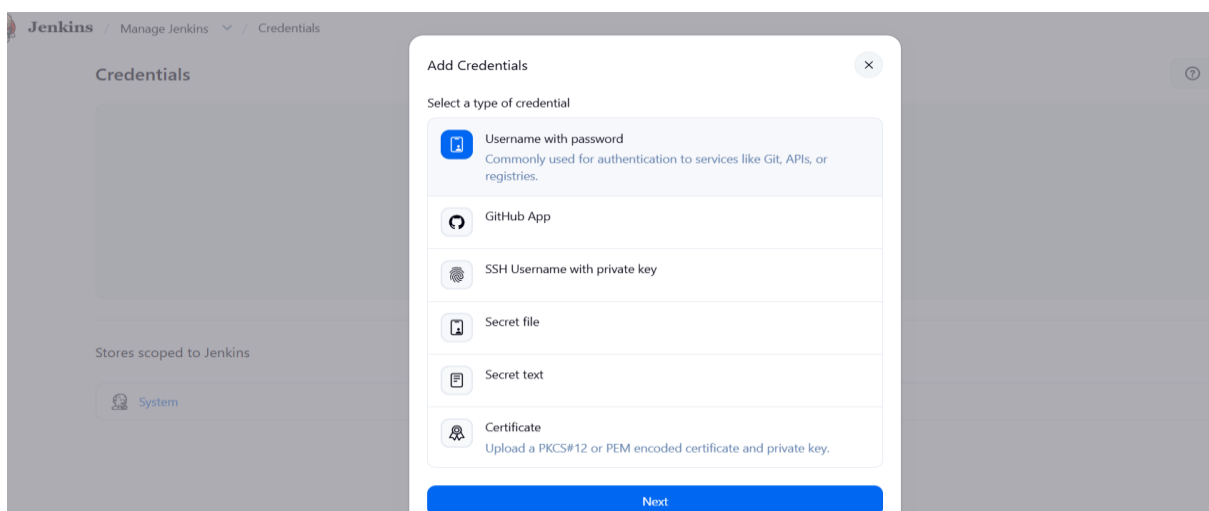
Creating GitHub Repo:



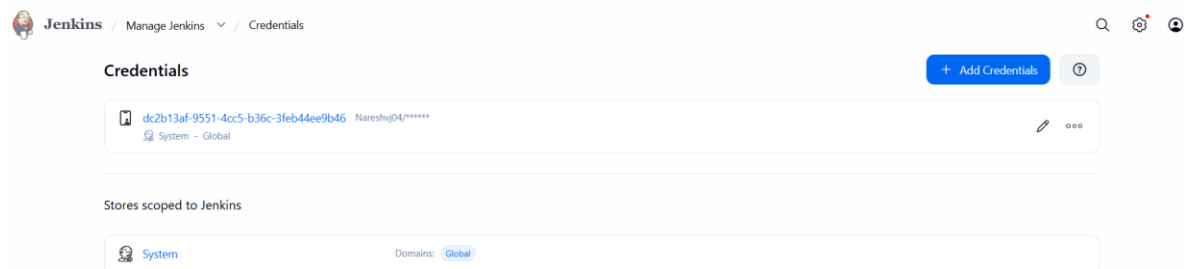
GitHub Webhook added:



Adding Github credentials to Jenkins:

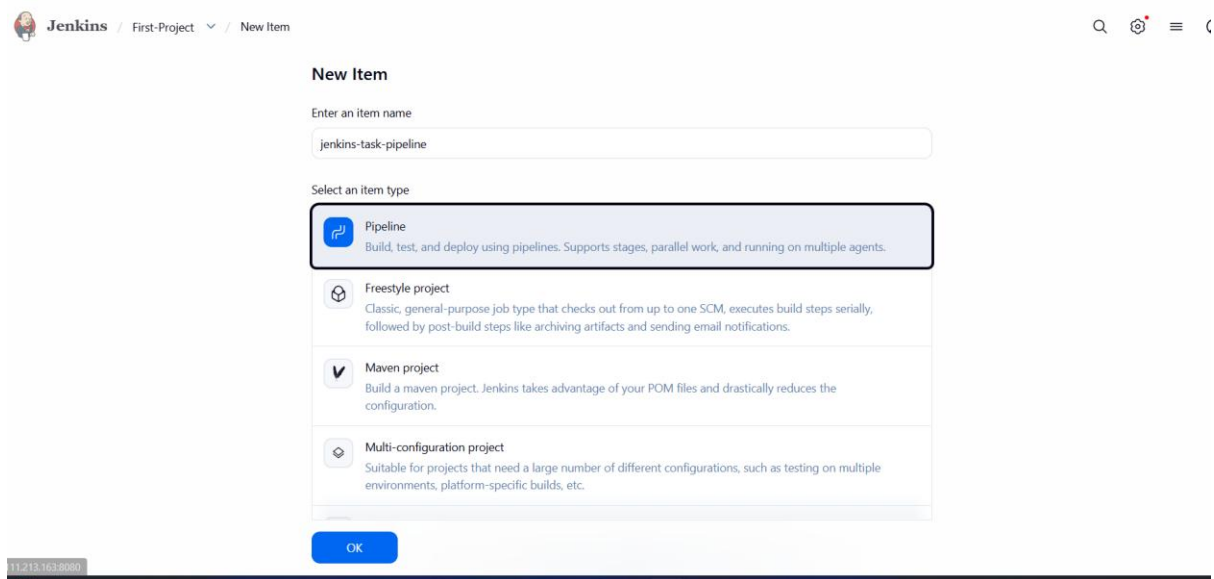


Credentials added:



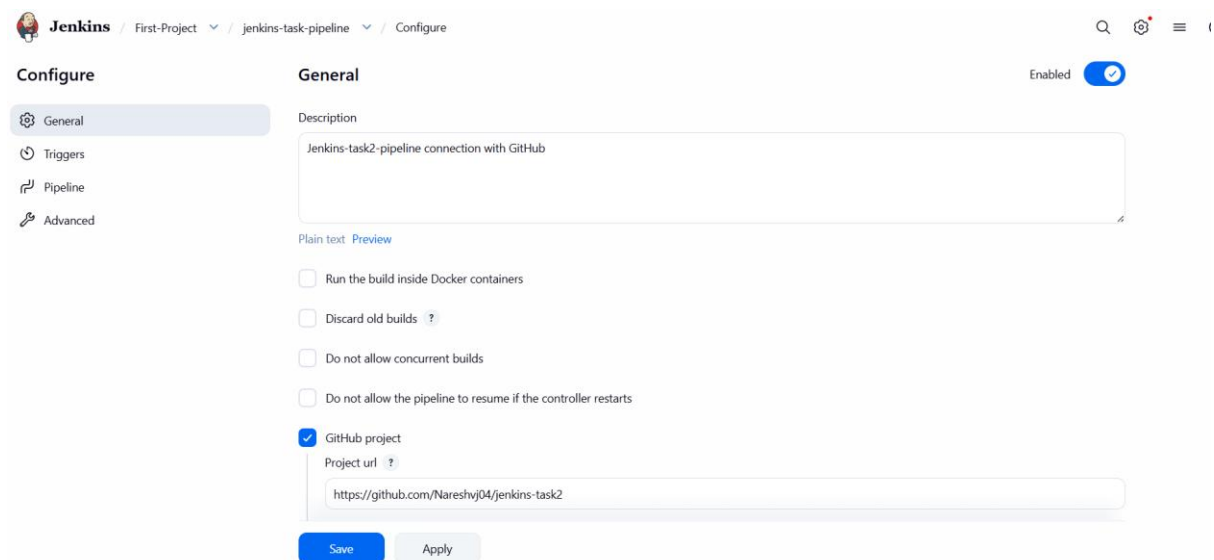
The screenshot shows the Jenkins 'Credentials' page. The breadcrumb navigation is 'Jenkins / Manage Jenkins / Credentials'. There is a search icon, a settings icon, and a user icon in the top right. A blue button '+ Add Credentials' and a help icon are at the top right. Below the header, a credential is listed with ID 'dc2b13af-9551-4cc5-b36c-3feb44ee9b46', name 'Nareshvj04/*****', and scope 'System - Global'. Below this, a section 'Stores scoped to Jenkins' shows 'System' as the store and 'Global' as the domain.

Creating Pipeline project:



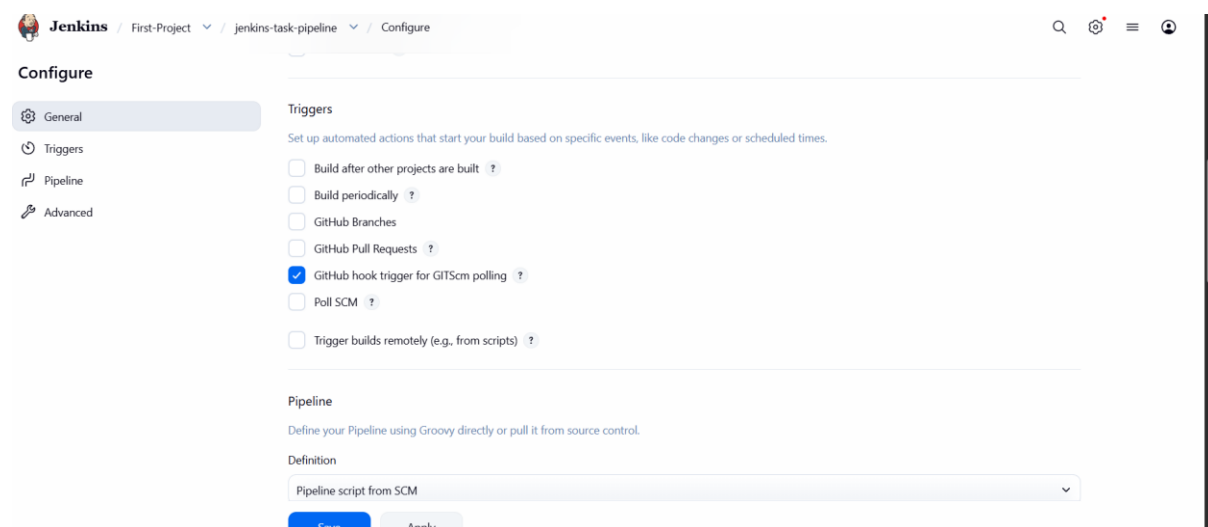
The screenshot shows the 'New Item' dialog in Jenkins. The breadcrumb navigation is 'Jenkins / First-Project / New Item'. There is a search icon, a settings icon, a menu icon, and a help icon in the top right. The 'Enter an item name' field contains 'jenkins-task-pipeline'. The 'Select an item type' section has four options: 'Pipeline' (selected), 'Freestyle project', 'Maven project', and 'Multi-configuration project'. The 'Pipeline' option is highlighted with a purple border. At the bottom, there is an 'OK' button.

Configure Jenkins with Github credentials:



The screenshot shows the 'Configure' page for a Jenkins Pipeline project. The breadcrumb navigation is 'Jenkins / First-Project / jenkins-task-pipeline / Configure'. There is a search icon, a settings icon, a menu icon, and a help icon in the top right. The 'Configure' section on the left has four tabs: 'General' (selected), 'Triggers', 'Pipeline', and 'Advanced'. The 'General' tab is active. The 'Description' field contains 'Jenkins-task2-pipeline connection with GitHub'. Below this, there are four checkboxes: 'Run the build inside Docker containers', 'Discard old builds', 'Do not allow concurrent builds', and 'Do not allow the pipeline to resume if the controller restarts'. The 'GitHub project' checkbox is checked. Below it, the 'Project url' field contains 'https://github.com/Nareshvj04/jenkins-task2'. At the bottom, there are 'Save' and 'Apply' buttons.

Trigger setup :



The screenshot shows the Jenkins configuration page for a project named 'jenkins-task-pipeline'. The 'Configure' tab is selected, and the 'Triggers' sub-tab is active. The 'Triggers' section contains a list of checkboxes for various build triggers. The 'GitHub hook trigger for GITScm polling' checkbox is checked. Below the triggers, the 'Pipeline' section is visible, showing the 'Definition' dropdown set to 'Pipeline script from SCM'. The 'Save' button is highlighted.

Configure

- General
- Triggers
- Pipeline
- Advanced

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☐ GitHub Branches ?
- ☐ GitHub Pull Requests ?
- ☒ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?
- ☐ Trigger builds remotely (e.g., from scripts) ?

Pipeline

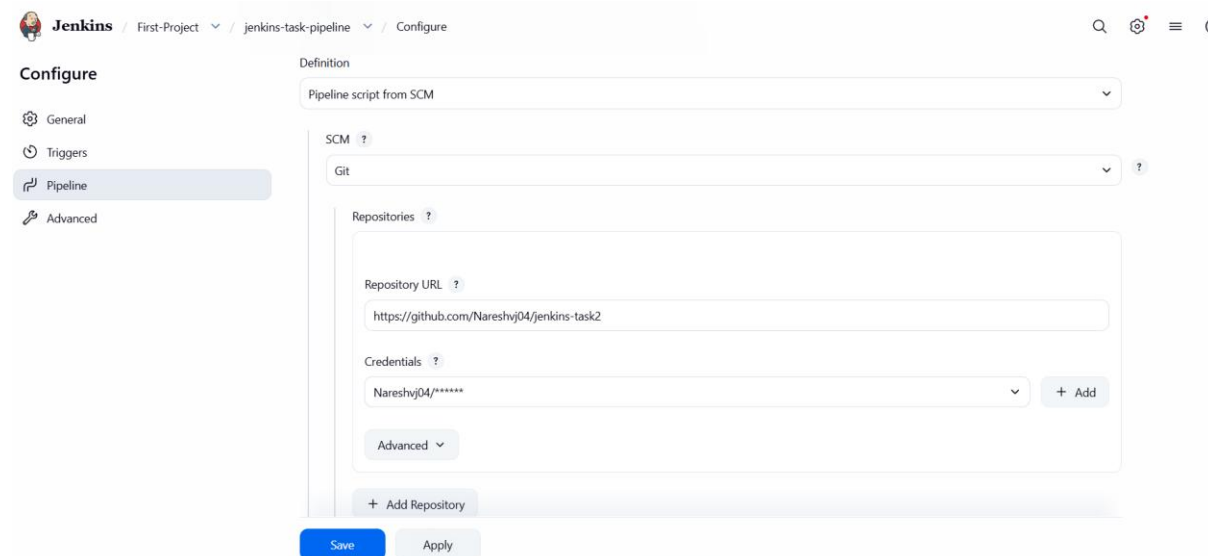
Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

Save Apply

SCM GitHub path:



The screenshot shows the Jenkins configuration page for the same project, but the 'Pipeline' sub-tab is active. The 'Definition' dropdown is set to 'Pipeline script from SCM'. The 'SCM' dropdown is set to 'Git'. The 'Repositories' section contains a form for adding a new repository. The 'Repository URL' field is filled with 'https://github.com/Nareshvj04/jenkins-task2'. The 'Credentials' dropdown is set to 'Nareshvj04/*****'. The 'Advanced' dropdown is expanded. The 'Add Repository' button is highlighted.

Configure

- General
- Triggers
- Pipeline
- Advanced

Definition

Pipeline script from SCM

SCM

Git

Repositories

Repository URL ?

https://github.com/Nareshvj04/jenkins-task2

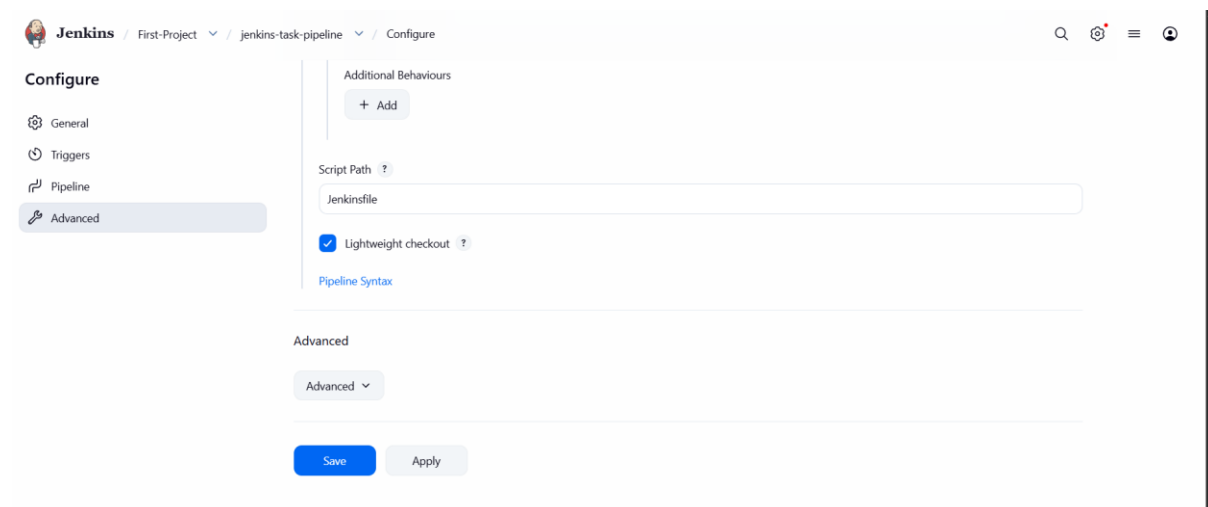
Credentials ?

Nareshvj04/*****

Advanced

+ Add Repository

Save Apply



The screenshot shows the Jenkins configuration page for the same project, but the 'Advanced' sub-tab is active. The 'Additional Behaviours' section contains a '+ Add' button. The 'Script Path' field is filled with 'Jenkinsfile'. The 'Lightweight checkout' checkbox is checked. The 'Pipeline Syntax' link is visible. The 'Advanced' dropdown is expanded. The 'Save' button is highlighted.

Configure

- General
- Triggers
- Pipeline
- Advanced

Additional Behaviours

+ Add

Script Path

Jenkinsfile

☒ Lightweight checkout ?

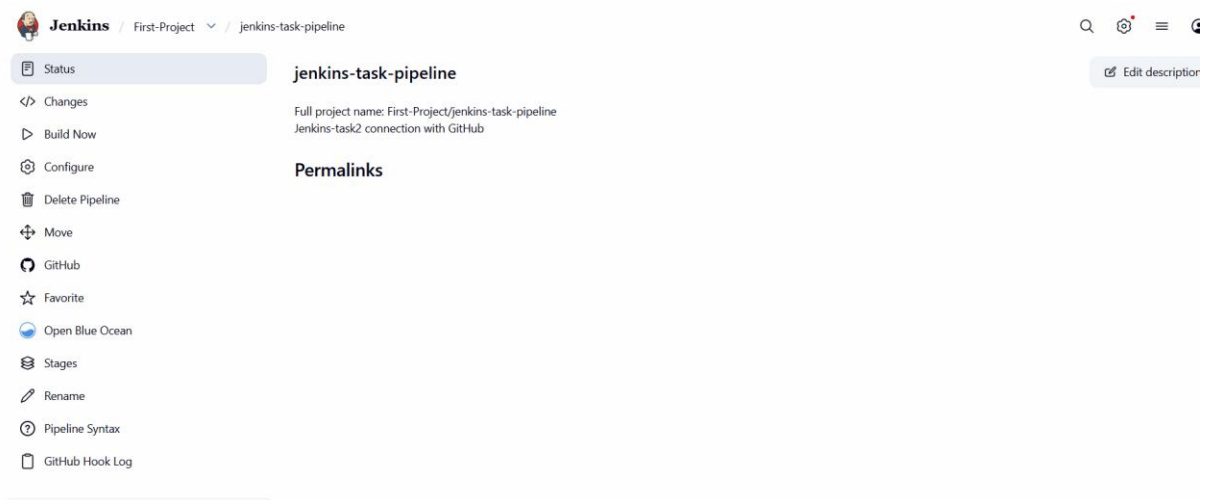
[Pipeline Syntax](#)

Advanced

Advanced

Save Apply

Project setup completed:



Creating Dockerfile:

```
root@jenkins-server:~/jenkins ✕ + ▾
FROM nginx:alpine
COPY . /usr/share/nginx/html
EXPOSE 80
~
~
~
~
~
~
```

Creating Jenkinsfile:

```
root@jenkins-server:~/jenkins ✕ + ▾
pipeline {
  agent {
    label 'built-in'
  }

  environment {
    IMAGE_NAME = "my-app"
    CONTAINER_NAME = "my-app-container"
    PORT = "8081"
  }

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main',
            url: 'https://github.com/Nareshvj04/jenkins-task2.git'
      }
    }

    stage('Build Image') {
      steps {
        sh 'docker build -t $IMAGE_NAME .'
      }
    }

    stage('Container Operation') {
      steps {
        sh '''
            docker stop $CONTAINER_NAME || true
            docker rm $CONTAINER_NAME || true
          '''
      }
    }
  }
}
```

2,1

```

    }
  }

  stage('Run New Container') {
    steps {
      sh '''
        docker run -d -p $PORT:80 --name $CONTAINER_NAME $IMAGE_NAME
      '''
    }
  }
}

post {
  success {
    emailxmt(
      subject: "SUCCESS: Job ${env.JOB_NAME} ${env.BUILD_NUMBER}",
      body: ""The build was successful!

BUILD SUCCESSFUL!

Job: ${env.JOB_NAME}
Build URL: ${env.BUILD_URL}
Build Number: ${env.BUILD_NUMBER}
""",
      to: "nareshvj0402@gmail.com"
    )
  }
}

```

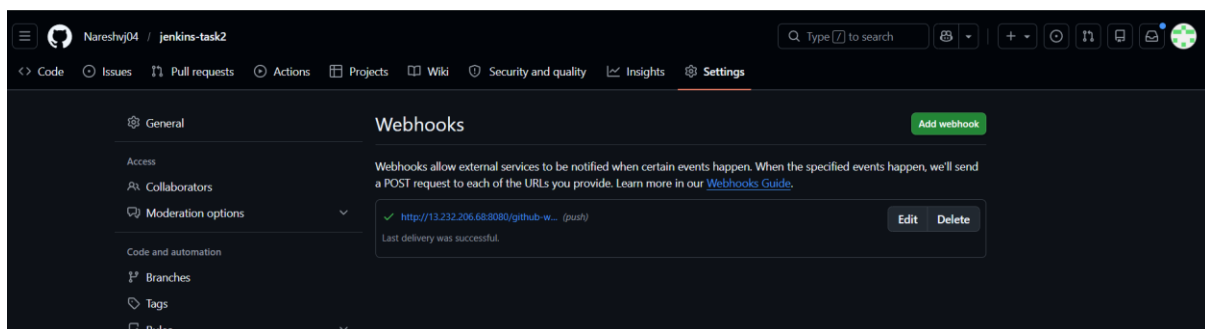
Changes pushed to the repo:

```

[root@jenkins-server jenkins-task2]# git add .
[root@jenkins-server jenkins-task2]# git commit -m "code changed"
[main 8009947] code changed
1 file changed, 1 insertion(+), 1 deletion(-)
[root@jenkins-server jenkins-task2]# git pull
Already up to date.
[root@jenkins-server jenkins-task2]# git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 283 bytes | 283.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:Nareshvj04/jenkins-task2.git
   9bfa34e..8009947  main -> main
branch 'main' set up to track 'origin/main'.

```

GitHub Webhook triggered:



Pipeline triggered:

 Jenkins

First-Project

jenkins-task-pipeline

#4

Search

Settings

Menu

User

Status

Changes

Console Output

Edit Build Information

Polling Log

Timings

Pipeline Overview

Open Blue Ocean

Thread Dump

Pause/resume

Replay

Pipeline Steps

Workspaces

Previous Build

Console

Download

Copy

View as plain text

```
Started by GitHub push by Nareshvj04
Obtained Jenkinsfile from git https://github.com/Nareshvj04/jenkins-task2
[Pipeline] Start of Pipeline
[Pipeline] node
Still waiting to schedule task
Waiting for next available executor
Running on Jenkins in /var/lib/jenkins/workspace/First-Project/jenkins-task-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential dc2b13af-9551-4cc5-b36c-3feb44ee9b46
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/First-Project/jenkins-task-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Nareshvj04/jenkins-task2 # timeout=10
Fetching upstream changes from https://github.com/Nareshvj04/jenkins-task2
> git --version # timeout=10
> git --version # 'git version 2.50.1'
using GIT_ASKPASS to set credentials

Checking out Revision 80099471226cb480b5fc162e19d45d4cf853f4ea (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 80099471226cb480b5fc162e19d45d4cf853f4ea # timeout=10
Commit message: "code changed"
> git rev-list --no-walk 9bfa34e36798d0f73859c159ab34c85aa7f7b3ec # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] git
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/First-Project/jenkins-task-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Nareshvj04/jenkins-task2.git # timeout=10
Fetching upstream changes from https://github.com/Nareshvj04/jenkins-task2.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/Nareshvj04/jenkins-task2.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 80099471226cb480b5fc162e19d45d4cf853f4ea (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
```

Jenkins

First-Project

jenkins-task-pipeline

#4

Search

Settings

Menu

User

#4 [internal] load build context

#4 transferring context: 15.24kB 0.0s done

#4 DONE 0.0s

#5 [1/2] FROM docker.io/library/nginx:alpine@sha256:dc48b7a872a79fb541ba5081d320b11b549231bc63ba465a7495afaa7d2ebcb8

#5 CACHED

#6 [2/2] COPY . /usr/share/nginx/html

#6 DONE 0.1s

#7 exporting to image

#7 exporting layers 0.0s done

#7 writing image sha256:e12b826dd9356aadce041d6a7207f229366870123a54bd2161eceed04e2f8bcf done

#7 naming to docker.io/library/my-app 0.0s done

#7 DONE 0.1s

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (Container Operation)

[Pipeline] sh

+ docker stop my-app-container

Error response from daemon: No such container: my-app-container

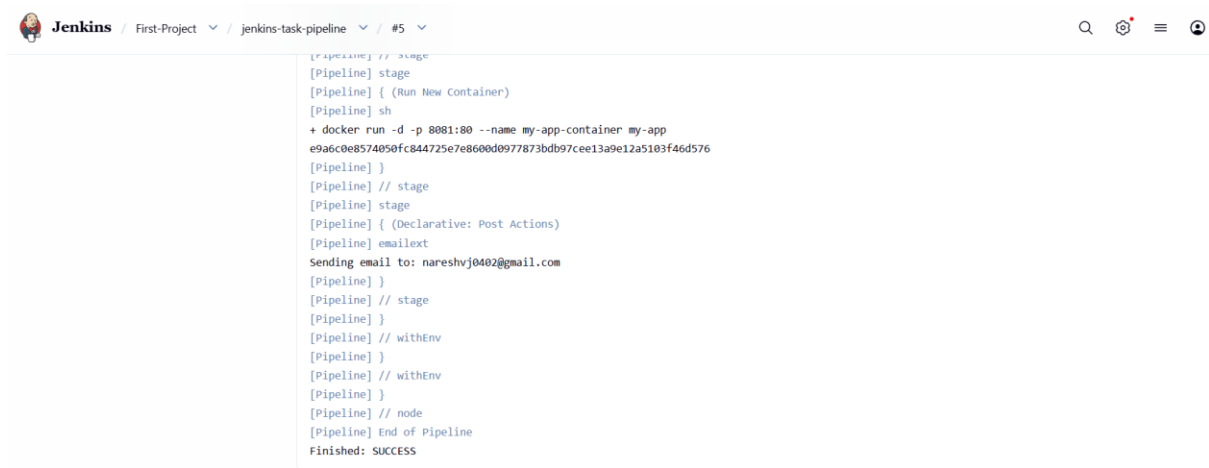
+ true

+ docker rm my-app-container

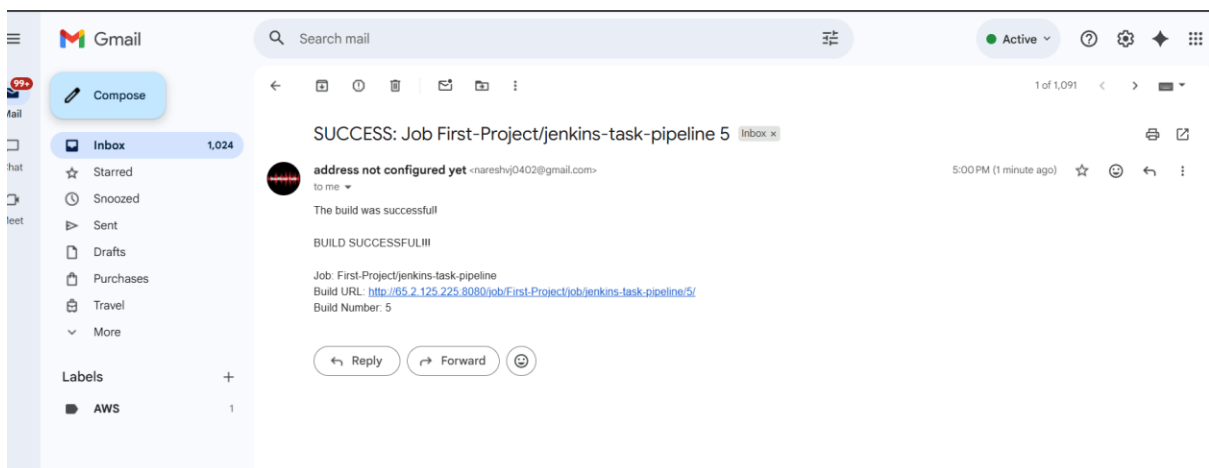
Error response from daemon: No such container: my-app-container

+ true

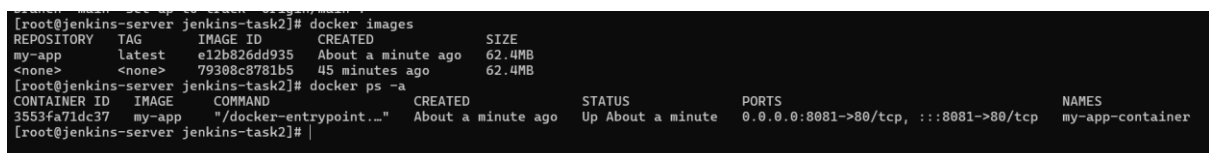
[Pipeline] }



Email received:



Docker image and contained created by Jenkins pipeline:



Nginx running on 8081:

